

Blog Paradigmas

Paradigmas Etiquetas HTML Estilos CSS Demo de Formulario Scripts JavaScript
Hoja de Excel

CONTENIDO

Conceptos básicos de JavaScript

Formas de enlazar JavaScript

Ámbito (Scope) en JavaScript

Variables y Tipos de Datos

| Tipos de datos existentes

Condicionales

Tipos de Funciones

[Descarga esta página](#) 

Conceptos básicos de JavaScript

JavaScript (JS) es un lenguaje de programación ligero, interpretado y orientado a objetos con funciones de primera clase. Es conocido principalmente como el lenguaje de scripting para las páginas web, permitiendo crear contenido dinámico, controlar multimedia, animar imágenes y mucho más.

Una de las características más importantes de JS es su **tipado dinámico** y su naturaleza asíncrona, lo que lo hace ideal para el desarrollo web moderno, incluyendo el desarrollo de interfaces de usuario complejas y la conexión con bases de datos.



El escudo amarillo que representa el ecosistema de JavaScript.

Formas de enlazar JavaScript con HTML

Existen tres formas principales de integrar código JavaScript en un documento HTML, dependiendo de la envergadura del script y la organización del proyecto:

1. **JavaScript en línea:** Se ejecuta directamente dentro de un atributo HTML (como `onclick`). No es recomendado para proyectos reales porque mezcla la estructura con la lógica.

```
<button onclick="alert('¡Hola!')">Haz clic</button>
```

2. **JavaScript interno:** Se coloca dentro de una etiqueta `<script>` en el `<head>` o antes del cierre del `<body>`. Útil para scripts pequeños o específicos de una sola página.
3. **JavaScript externo:** Es la mejor práctica. Se crea un archivo `.js` independiente y se enlaza mediante el atributo `src`. Permite reutilizar la lógica y mantener el código ordenado.

```
<script src="app.js"></script>
```

Ámbito (Scope) en JavaScript

Antes de entrar en el tema de la declaración de variables, es importante entender el concepto de **Ámbito... (Scope)** en JavaScript. El ámbito define el contexto en el que las variables y funciones son válidas y pueden ser referenciadas.

Dependiendo de cómo declares una variable (con `var`, `let` o `const`), JavaScript le asignará un tipo de ámbito diferente: Ámbito Global, Ámbito de Función o Ámbito de Bloque.

Ámbito Global (Global Scope)

Una variable tiene ámbito global si se declara en el nivel superior del documento, fuera de cualquier función o bloque. Estas variables son accesibles y modificables desde cualquier parte del código a lo largo de todo el programa. Aunque pueda parecer conveniente al principio, abusar del ámbito global se considera una mala práctica porque aumenta el riesgo de sobrescribir variables accidentalmente (colisión de nombres) y hace que el código sea difícil de depurar.

Ámbito de Función (Function Scope)

Cuando declaras una variable dentro de una función, esta pertenece de forma exclusiva al ámbito de esa función. Significa que solo puede ser leída y modificada desde dentro de la función misma. Cualquier bloque de código externo a la función será completamente ciego a la existencia de esa variable.

Ámbito de Bloque (Block Scope)

Un "bloque" se define como cualquier fragmento de código encerrado entre llaves `{}`, como ocurre dentro de un condicional `if`, un bucle `for` o un `while`. Las variables con ámbito de bloque nacen y mueren estrictamente dentro de esas llaves, proporcionando un control granular y evitando que variables temporales "contaminen" el resto de la función.

```
// Ejemplo práctico de los distintos ámbitos
let mensajeGlobal = "Soy global"; // Accesible en todos lados
```

```
function demostrarScope() {  
  var mensajeFuncion = "Soy exclusivo de la función";  
  
  if (true) {  
    let mensajeBloque = "Solo existo en el if";  
    console.log(mensajeBloque); // Se imprime correctamente  
  }  
  
  console.log(mensajeFuncion); // Se imprime correctamente  
  
  // console.log(mensajeBloque);  
  // ¡ERROR! ReferenceError: mensajeBloque is not defined  
}  
  
demostrarScope();  
// console.log(mensajeFuncion);  
// ¡ERROR! No se puede acceder a la variable desde fuera de la función
```

Variables y Tipos de Datos

En JavaScript, una variable es un contenedor para almacenar datos. Gracias al tipado dinámico, la variable se amolda al tipo de dato que se le asigna en ese momento, y puede cambiar de tipo posteriormente sin generar errores. Existen tres formas de declarar variables:

- **var**: La forma antigua. Tiene un alcance (scope) global o de función, lo que puede causar bugs.
- **let**: La forma moderna. Tiene alcance de bloque, lo que la hace más segura.
- **const**: Para declarar valores que no deben ser reasignados.

Tipos de datos que existen

JavaScript maneja varios tipos de datos primitivos y estructurales:

- **String**: Cadenas de texto.
- **Number**: Números, ya sean enteros o decimales.

- **Boolean:** Valores lógicos (`true` o `false`).
- **Null:** Representa la ausencia intencional de cualquier valor.
- **Undefined:** Una variable que ha sido declarada pero aún no se le ha asignado un valor.
- **Object:** Colecciones complejas de datos (como un registro de estudiantes).

```
// Ejemplo de tipado dinámico
let datoEstudiante = "Kelvin"; // Inicialmente es un String
datoEstudiante = 2026; // Ahora se amolda y es un Number (Año de graduación)
datoEstudiante = true; // Ahora es un Boolean (¿Está inscrito?)

// Objeto
const registro = {
  nombre: "Perla",
  carrera: "Informática"
};
```

Condicionales

Las estructuras condicionales permiten ejecutar diferentes bloques de código basándose en el cumplimiento de ciertas condiciones (expresiones que retornan un booleano).

- **if / else:** La estructura básica de control.
- **switch:** Evalúa una expresión comparándola con múltiples casos. Excelente para menús o estados definidos.
- **Operador Ternario:** Una forma compacta de escribir un `if...else`. Se compone de una condición seguida de un signo de interrogación (`?`), la expresión si es verdadera, dos puntos (`:`) y la expresión si es falsa.

```
let notaProyecto = 18;

if (notaProyecto >= 15) {
  console.log("Aprobado con honores.");
}
```

```
} else if (notaProyecto >= 10) {  
  console.log("Aprobado.");  
} else {  
  console.log("Reprobado.");  
}  
  
// Operador Ternario  
let estado = (notaProyecto >= 10) ? "Pasa la materia" : "Debe repetir";
```

Tipos de Funciones

Las funciones son bloques de código diseñados para realizar una tarea particular. En JS, las funciones son ciudadanos de primera clase.

1. **Funciones Declarativas:** Se definen usando la palabra clave `function`. Tienen "hoisting" (pueden ser llamadas antes de ser declaradas en el código).
2. **Funciones Expresivas (Anónimas):** Se asignan a una variable. No se pueden invocar antes de su inicialización.
3. **Arrow Functions (Funciones Flecha):** Introducidas en ES6, ofrecen una sintaxis más corta y no tienen su propio contexto para `this`, lo que las hace ideales para callbacks (muy usadas en frameworks como React).

```
// 1. Declarativa  
function saludar() {  
  return "Hola a todos";  
}  
  
// 2. Expresiva  
const despedir = function() {  
  return "Hasta luego";  
};  
  
// 3. Función Flecha (Arrow Function)  
const calcularPromedio = (a, b) => {  
  return (a + b) / 2;  
};
```

```
// Flecha con retorno implícito (una sola línea)  
const multiplicar = (x, y) => x * y;
```



Blog hecho por Kelvin Bolívar. Encuentra mis otros proyectos en mi pagina de [Github](#)